

APUNTS LAB1 GRÀFICS

1·VERTEX SHADER

- Passar a Normalitzades Disp i després a Window:

```
//          (4)          (4)          (4)          (3)          (2)
//object -- MT --> world space -- VT --> eye space -- PT --> Clipping -- dividir --> NDC -- formula --> window :)
vec4 P_clip = modelViewProjectionMatrix * vec4(vertex, 1.0); //obtenim clipping
vec3 P_ndc = P_clip.xyz / P_clip.w; //obtenim normalitzades de dispositiu
vec2 P_win = ((P_ndc.xy + 1) / 2) * viewport; //obtenim window
```

- Gris: posar el color com a N.z
- Contortion per a veure tras + rot + tras
- Smoothstep:
 // 0 si vertex.y <= 1.45, 1 si vertex.y >= 1.55
 float t = smoothstep(1.45, 1.55, vertex.y);
- Mix (interpolació lineal):
 vec3 p = mix(A,B,t) //interpola de A a B amb t = [0,1]
- Desplaçar un punt sobre un altre o en la direcció de un altre:
 // desplace Peye en la direcció de la normal + no se que
 vec3 vertexPprim = Peye + (N * 0.03 * distance(boundingBoxMax, boundingBoxMin));
- Mirar uniforms en Laab1 el pdf Exercicis
- Calcular vector que va de un punt a un altre: vec3 u = destí - origen
- clamp(0.0, 1.0, v) fa que v estigui entre [0,1]
- $A * \sin(2\pi * f * t + \Theta)$ // la f ens cada quant repetim animació o similar
- Rotar sobre un eix abans cal restar pivot, rotar, sumar pivot (dolphin)
- Producte escalar és el dot(); //dot calcula el cos de l'angle que formen els dos vec!

2·ILUMINACIÓ

- Uniforms:

```
uniform vec4 lightAmbient;
uniform vec4 lightDiffuse;
uniform vec4 lightSpecular;
uniform vec4 lightPosition;
```

```
// (sempre estarà en eye space)
uniform vec4 matAmbient;
uniform vec4 matDiffuse;
uniform vec4 matSpecular;
uniform float matShininess;
```

· Llum phong al vertex shader:

```
vec3 P_eye = (modelViewMatrix * vec4(vertex, 1.0)).xyz; //punt en eye space
vec3 V = normalize(-P_eye); //vector unitari vertex cap a camera (0,0,0) - P_eye
vec3 N = normalize(normalMatrix * normal); //vector normal com sempre
vec3 L = normalize(-P_eye); //[(0,0,0) - P_eye] o [lightPosition - P_eye]
vec3 R = reflect(-L,N); //vector reflexio
```

```
vec3 colorLlum = vec3(matAmbient*lightAmbient + matDiffuse*lightDiffuse*max(0.0,dot(N,L)) +
matSpecular*lightSpecular*pow(max(0.0, dot(R,V)),matShininess));
```

· Llum phong al fragment shader:

```
// passem des de VS el vertex i normal sense modificar-los
vec3 P_eye = (modelViewMatrix * vec4(v, 1.0)).xyz; //punt en eye space
vec3 V = normalize(-P_eye); //vector unitari vertex cap a camera (0,0,0) - P_eye
vec3 N = normalize(normalMatrix * n); //vector normal com sempre
vec3 L = normalize(-P_eye); //[(0,0,0) - P_eye] o [lightPosition - P_eye]
vec3 R = reflect(-L,N); //vector reflexio
```

```
vec3 colorLlum = vec3(matAmbient*lightAmbient + matDiffuse*lightDiffuse*max(0.0,dot(N,L)) +
matSpecular*lightSpecular*pow(max(0.0, dot(R,V)),matShininess));
```

o utilitzar funció:

```
vec4 light(vec3 N, vec3 V, vec3 L)
{
    N = normalize(N); V = normalize(V); L = normalize(L);
    vec3 R = normalize( 2.0*dot(N,L)*N-L );
    float NdotL = max( 0.0, dot( N,L ) );
    float RdotV = max( 0.0, dot( R,V ) );
    float Idiff = NdotL;
    float Ispec = 0;
    if (NdotL>0) Ispec = pow( RdotV, matShininess );
    return
        matAmbient * lightAmbient +
        matDiffuse * lightDiffuse * Idiff +
        matSpecular * lightSpecular * Ispec;
}
```

· Gradients al lab 2.

3·SHADERS

- Normalment cal fer `in vec2 vtxCoord` i `uniform sampler2D nomText` en FS.
- Si vols repetir x vegades una textura cal fer: `x \* vtxCoord` on vulguis.
- Fer que algo evolucioni amb el temps: `vtxCoord + (time \* speed)`
- FPS i com escollir un frame: (de explosion)

- Si tenim FPS el que hem de fer per saber quin frame toca és:

```
int frame = int(FPS * time) % n_frames // o usar time / segons_donats
int row = frame / n_files // es / porque anem de esq a dre, si es amunt a abais usem %
int col = frame % / n_cols // n files i n cols poden ser el mateix! n_files si mirem col per col

float offsetX = float(col) * (1.0/8.0); // entre num cols
float offsetY = 5.0 - float(row) * (1.0/6.0); //volem anar cap a dalt
// canvi_base + quin_quadre
vec2 vtxCoordNou = vec2(vtxCoord.s * (1.0/8.0) + offsetX, vtxCoord.t * (1.0/6.0) + offsetY);

///// RECORDA, si repetim la textura cal usar fract(vtxCoord.algo) per fer-ho bé!!!
```

- Escollir tile sense temps: (com pacman o invaders)

```
vec2 fant = vec2(0.0, 0.0); //calculem offsets de cada tile (on comencen)
vec2 pacm = vec2(1.0 * 1.0/6.0, 0.0);
vec2 noth = vec2(2.0 * 1.0/6.0, 0.0);
vec2 stra = vec2(3.0 * 1.0/6.0, 0.0);
vec2 curv = vec2(4.0 * 1.0/6.0, 0.0);
vec2 pind = vec2(5.0 * 1.0/6.0, 0.0);
```

```
int n = 10; //mida matriu nxn
vec2 coordMat = fract(vtxCoord * n); //coordText dins de la cela, va de 0.0 a 0.99
vec2 cela = floor(vtxCoord * n); //numero de cela entre 0 i 9
```

```
vec2 escalat = vec2(coordMat.x / 6.0, coordMat.y); //nova base, tocar y si cal
```

```
vec2 offseet;
```

```
if ((cela.x == 0 || cela.x == 9) && (cela.y == 0 || cela.y == 9)) offseet = curv;
else if (cela.x == 0 || cela.x == 9) offseet = stra;
else if (cela.y == 0 || cela.y == 9) offseet = stra;
else offseet = pacm;
```

```
fragColor = texture(colormap, escalat + offseet);
```

- Rotar mirar players: fer coordNoves.y = 1.0 - coordNoves.y
- Fer cercles:

```
// dist entre dos punts
float d = distance(vtxCoord, vec2(0.5,0.5)); //distancia del centre al vtxCoord
// radi
if (step(0.2,d) == 0.0) fragColor = vec4(1.0, 0.0, 0.0, 0.0);
else fragColor = vec4(1.0, 1.0, 1.0, 1.0);
```

- Fer dibuixos amb cercles mirar halloween2, bàsicament anar fent cercles grans i dins d'aquests fer de petits :)

· Rotar textura:

Cal rotar abans d'aplicar escalat!!!

```
vec2 coordRot = coordMat;
```

coordRot = 0° (Original): `vec2(coordMat.x, coordMat.y)`

coordRot = 90° (Horario): `vec2(coordMat.y, 1.0 - coordMat.x)`

coordRot = 180°: `vec2(1.0 - coordMat.x, 1.0 - coordMat.y)`

coordRot = 270° (Antihorario): `vec2(1.0 - coordMat.y, coordMat.x)`

```
vec2 escalar = ...;
```